

UartToCan_test 工程架构与数据流分析

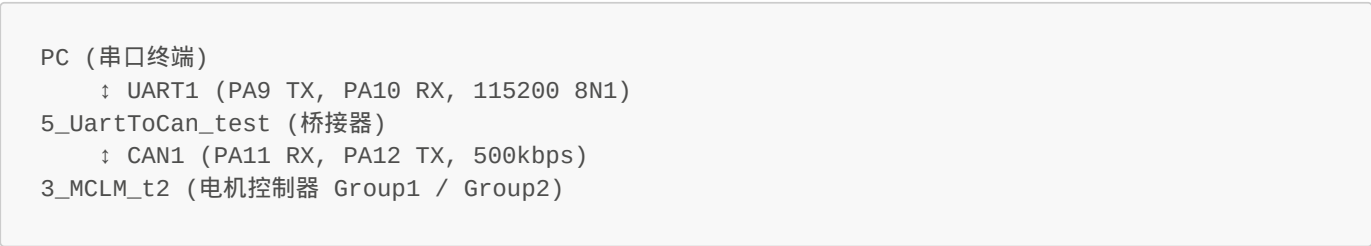
硬件资源

外设	引脚	速率	用途
USART1	PA9(TX), PA10(RX)	115200 8N1	与 PC 通信（协议+调试）
USART2	PA2(TX), PA3(RX)	115200 8N1	已初始化，应用层未使用
CAN1	PA11(RX), PA12(TX)	500kbps	与电机控制器通信
PC13	板载 LED	-	心跳指示 (300ms 翻转)

CAN 波特率计算：APB1=36MHz, Prescaler=4, BS1=13TQ, BS2=4TQ → 36MHz / 4 / (1+13+4) = 500kbps

系统概述

5_UartToCan_test 是一个 **UART ↔ CAN 双向桥接器**，运行于 STM32F103C8T6，连接 PC 与两组电机控制器 (每组含转向+动力两台电机)。

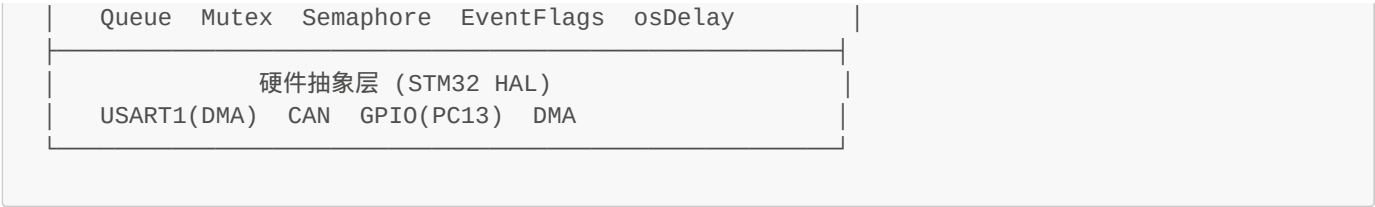


一、工程文件职能总览

App/	
├─ app_config.h	— 协议常量、数据结构类型定义（全局类型中枢）
├─ app_globals.h	— 跨模块共享句柄的 extern 声明
├─ app_includes.h	— 统一总头文件（单点包含入口）
├─ ring_buffer.h/c	— 无锁环形缓冲区（ISR↔Task 数据桥）
└─ app_task.h/c	— 四个 FreeRTOS 任务的全部实现
doc/	
├─ trae.md	— 架构与数据流文档（主设计文档）
├─ claude.md	— 代码审查优化归档（P0/P1/P2 问题记录）
├─ fix1_UartToDma.md	— UART TX 阻塞→DMA 改造专题记录
├─ result.md	— 项目目标完成状态
├─ goal.md	— 演进规划与验证要点
└─ Now_all.md	— 当前状态汇总（本文件）

二、分层架构





三、各文件职能详解

3.1 app_config.h — 配置与类型定义

元素	类型	作用
FRAME_SOF 0xAA	宏	自定义协议帧头标志字节
CAN_RX_QUEUE_SIZE 16	宏	CAN 接收队列容量
UART_TO_CAN_QUEUE_SIZE 16	宏	UART → CAN 转发队列容量
Command_ID_t	enum	指令集：CMD_SET_SPEED / CMD_GET_STATE / CMD_SET_MODE / CMD_ESTOP
App_UART_Message_t	struct	UART 解析结果 (id + cmd + len + data[8])，入 uartToCanQueue
App_CAN_Message_t	struct	CAN 接收报文 (id + len + data[8])，入 canRxQueue
ParserState_t	enum	状态机五个状态：STATE_WAIT_SOF / STATE_WAIT_CMD / STATE_WAIT_ID / STATE_WAIT_LEN / STATE_WAIT_DATA

App_UART_Message_t 中 uint32_t id 优先排列，结构体大小 16 字节，无填充浪费。

3.1.1 自定义 UART 协议帧格式

桥接器与 PC 间使用以下帧格式通信：

字段	大小	说明
SOF	1B	固定 0xAA
CMD	1B	指令 (Command_ID_t)
ID	4B LE	CAN ID (小端，支持 29 位扩展帧)
LEN	1B	Data 长度 (0~8)
DATA	LEN B	CAN 数据负载

3.1.2 CAN ID 定义

桥接器同时管理两组电机控制器，CAN ID 编码规律：

- 0x1xx = 控制指令（桥接器 RX）
- 0x2xx = 状态查询（桥接器 RX）
- 0x3xx = 状态反馈（桥接器 TX）
- x23/x24 = Group 2（转向/动力）
- x25/x26 = Group 1（转向/动力）

宏	ID	方向	组	用途
CAN_MOTOR_TURN_CMD_STDID_G2	0x123	RX	G2	转向电机控制指令
CAN_MOTOR_POWER_CMD_STDID_G2	0x124	RX	G2	动力电机控制指令

宏	ID	方向	组	用途
CAN_MOTOR_TURN_CMD_STATUS_STDID_G2	0x223	RX	G2	转向电机状态查询
CAN_MOTOR_POWER_CMD_STATUS_STDID_G2	0x224	RX	G2	动力电机状态查询
CAN_MOTOR_TURN_STATUS_STDID_G2	0x323	TX	G2	转向电机状态反馈
CAN_MOTOR_POWER_STATUS_STDID_G2	0x324	TX	G2	动力电机状态反馈
CAN_MOTOR_TURN_CMD_STDID_G1	0x125	RX	G1	转向电机控制指令
CAN_MOTOR_POWER_CMD_STDID_G1	0x126	RX	G1	动力电机控制指令
CAN_MOTOR_TURN_CMD_STATUS_STDID_G1	0x225	RX	G1	转向电机状态查询
CAN_MOTOR_POWER_CMD_STATUS_STDID_G1	0x226	RX	G1	动力电机状态查询
CAN_MOTOR_TURN_STATUS_STDID_G1	0x325	TX	G1	转向电机状态反馈
CAN_MOTOR_POWER_STATUS_STDID_G1	0x326	TX	G1	动力电机状态反馈

3.1.3 CAN 状态帧格式（8 字节）

目标机上报的电机状态帧解码格式（0x323/0x324/0x325/0x326）：

偏移	类型	字段	对应 Motor_t 字段
[0-1]	int16 LE	current_logic_speed	Motor_t.current_logic_speed
[2-3]	uint16 LE	accumulated_ticks	Motor_t.accumulated_ticks (低16位)
[4-5]	int16 LE	pwm_output	Motor_t.pwm_output
[6]	int8	target_logic_speed	Motor_t.target_logic_speed
[7]	uint8	flags	Motor_t.flags

flags 位定义： 0x01=堵转(STALL), 0x02=PID饱和(SATURATED)

目标机 Motor_t 结构体定义于 3_MCLM_t2/App/config/app_globals.h，桥接器仅做解码转发，不持有结构体实例。

3.2 app_globals.h — 全局句柄声明

变量	类型	作用
hcan	CAN_HandleTypeDef	HAL CAN 外设句柄
huart1	UART_HandleTypeDef	USART1（调试/协议口）句柄
huart2	UART_HandleTypeDef	USART2（备用）句柄
canRxQueueHandle	osMessageQueueId_t	CAN ISR → CanRxProcess 任务的消息队列
uartToCanQueueHandle	osMessageQueueId_t	ProtocolParser → UartToCan 任务的消息队列
uart1_tx_mutexHandle	osMutexId_t	保护 UART1 TX 通道，防多任务并发乱码
uart1_tx_semHandle	osSemaphoreId_t	DMA 传输完成信号量（ISR 释放，任务等待）
uart1_rx_eventHandle	osEventFlagsId_t	ISR 通知 ProtocolParser 有新数据到来
UART1_RX_FLAG 0x01U	宏	事件标志位掩码

3.3 ring_buffer.h / ring_buffer.c — 无锁环形缓冲区

设计精髓：利用 `uint8_t` 自然溢出实现无锁 SPSC（单写单读）缓冲。

元素	说明
<code>UART_RX_BUFFER_SIZE 256</code>	固定 256， <code>uint8_t</code> 溢出回绕 $\equiv \% 256$ ，无需取模运算
<code>volatile uint8_t head</code>	ISR 写， <code>volatile</code> 防止编译器寄存器缓存
<code>volatile uint8_t tail</code>	任务读，单字节操作在 Cortex-M3 天然原子
<code>ring_buffer_init()</code>	<code>head = tail = 0</code> ，清空缓冲区
<code>ring_buffer_put()</code>	ISR 调用： <code>next_head == tail</code> 则满，返回 <code>false</code> 丢弃
<code>ring_buffer_get()</code>	任务调用： <code>head == tail</code> 则空，返回 <code>false</code>

```
buffer[0..255]: [ ... data bytes ... ]
                ↑tail(读)    ↑head(写)
ISR  向 head 写入，head++ (uint8_t 自动回绕)
Task 从 tail 读取，tail++ (uint8_t 自动回绕)
满判断：(head + 1) == tail
空判断：head == tail
```

3.4 `app_task.h` — 任务接口声明

纯声明四个任务函数，供 `freertos.c`（CubeMX 生成）在创建任务时注册：

函数	绑定任务	CMSIS 优先级	栈大小
<code>ProtocolParser_Task_Run()</code>	<code>ProtocolParser-Ta</code>	<code>osPriorityNormal1 (53)</code>	1024 B
<code>UartToCan_Task_Run()</code>	<code>UartToCan-Ta</code>	<code>osPriorityNormal (52)</code>	2048 B
<code>CanRxProcess_Task_Run()</code>	<code>CanRxProcess-Ta</code>	<code>osPriorityNormal (52)</code>	2048 B
<code>Heartbeat_Task_Run()</code>	<code>Heartbeat-Ta</code>	<code>osPriorityLow (8)</code>	256 B

3.5 `app_task.c` — 核心任务实现

静态辅助函数：`uart1_send(buf, len)` — DMA 发送封装

```
调用方
|  osMutexAcquire(uart1_tx_mutexHandle)  ← 独占 TX 通道
|  memcpy → uart1_tx_dma_buf[128]       ← 拷贝到静态缓冲 (DMA 需持续有效)
|  HAL_UART_Transmit_DMA()              ← 启动 DMA，立即返回
|  osSemaphoreAcquire(uart1_tx_semHandle) ← 挂起，等待 TxCplt 回调
↓                                         ↑ ISR 中 osSemaphoreRelease()
osMutexRelease(uart1_tx_mutexHandle)    ← 释放通道
```

关键变量：

变量	说明
<code>uart1_tx_dma_buf[128]</code>	<code>static</code> 静态数组，生命周期贯穿整个 DMA 传输
<code>UART1_TX_DMA_BUF_SIZE 128</code>	最大单次发送长度限制

任务①：ProtocolParser_Task_Run() — 协议解析（最高优先级）

状态机驱动，局部关键变量：

变量	类型	作用
state	ParserState_t	当前状态机状态
current_msg	App_UART_Message_t	正在组装中的帧
byte_received	uint8_t	从环形缓冲区取出的单字节
id_byte_count	uint8_t	已接收的 CAN_ID 字节数（累计到 4）
data_idx	uint8_t	已接收的数据负载字节数

状态机逻辑（每次取出 1 字节驱动）：

STATE_WAIT_SOF → 检测 0xAA → STATE_WAIT_CMD

STATE_WAIT_CMD → 存 cmd → STATE_WAIT_ID

STATE_WAIT_ID → 小端拼接 4 字节 id → STATE_WAIT_LEN

STATE_WAIT_LEN → 检查 len ≤ 8 → STATE_WAIT_DATA (len==0 直接入队)

STATE_WAIT_DATA → 填 data[] → 满足 len 后 osMessageQueuePut → STATE_WAIT_SOF

任意状态收到 0xAA → 视为新帧开始，重置并跳回 STATE_WAIT_CMD（容错）

阻塞机制：缓冲区为空时调用 `osEventFlagsWait`（10 ms 超时），ISR 写入后立即 `osEventFlagsSet` 唤醒，近零延迟。

任务②：UartToCan_Task_Run() — UART → CAN 转换

局部变量	作用
uart_msg (App_UART_Message_t)	从 uartToCanQueueHandle 取出的消息
tx_header (CAN_TxHeaderTypeDef)	HAL CAN 发送帧头（IDE / RTR / DLC / ID）
tx_mailbox (uint32_t)	HAL 分配的 CAN 发送邮箱编号
dbg_buffer[128]	sprintf 诊断字符串暂存

帧类型自动判断：

```
if (uart_msg.id > 0x7FF) // 扩展帧 29-bit
    tx_header.IDE = CAN_ID_EXT; tx_header.ExtId = uart_msg.id;
else // 标准帧 11-bit
    tx_header.IDE = CAN_ID_STD; tx_header.StdId = uart_msg.id;
```

任务③：CanRxProcess_Task_Run() — CAN → UART 解码转发

局部变量	作用
rx_can_msg (App_CAN_Message_t)	从 canRxQueueHandle 取出的 CAN 报文
tx_buffer[128]	格式化输出字符串缓冲区

两条输出路径：

① 电机状态帧 (ID 匹配 0x323/0x324/0x325/0x326) — 按新格式解码:

```
int16_t  current_speed = data[0..1]; // int16 LE
uint16_t accum_ticks   = data[2..3]; // uint16 LE
int16_t  pwm           = data[4..5]; // int16 LE
int8_t   target_speed  = data[6];    // int8
uint8_t  flags         = data[7];    // flags
```

输出示例:

```
CAN RX | TURN(G2) STATUS | curr=123 accum=456 pwm=789 target=50 flags=0x00
CAN RX | POWER(G1) STATUS | curr=-50 accum=1234 pwm=-200 target=-30 flags=0x01 STALL
```

② 非状态帧 — 原始 hex dump:

```
CAN RX | ID: 0x123 | DLC: 8 | Data: AA 01 00 00 00 08 64 00
```

两种路径均通过 `uart1_send()` 加锁发送。

任务④: `Heartbeat_Task_Run()` — 心跳

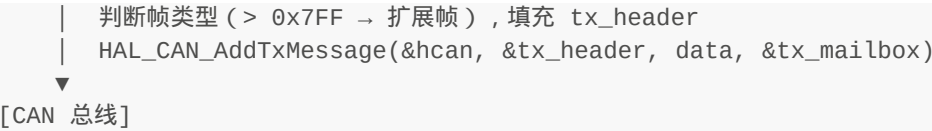
```
HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13); // 翻转板载 LED
osDelay(300);                             // 300 ms 周期
```

系统存活指示: LED 停止闪烁 = 任务死锁或崩溃。

四、完整数据流图

4.1 上行: UART → CAN

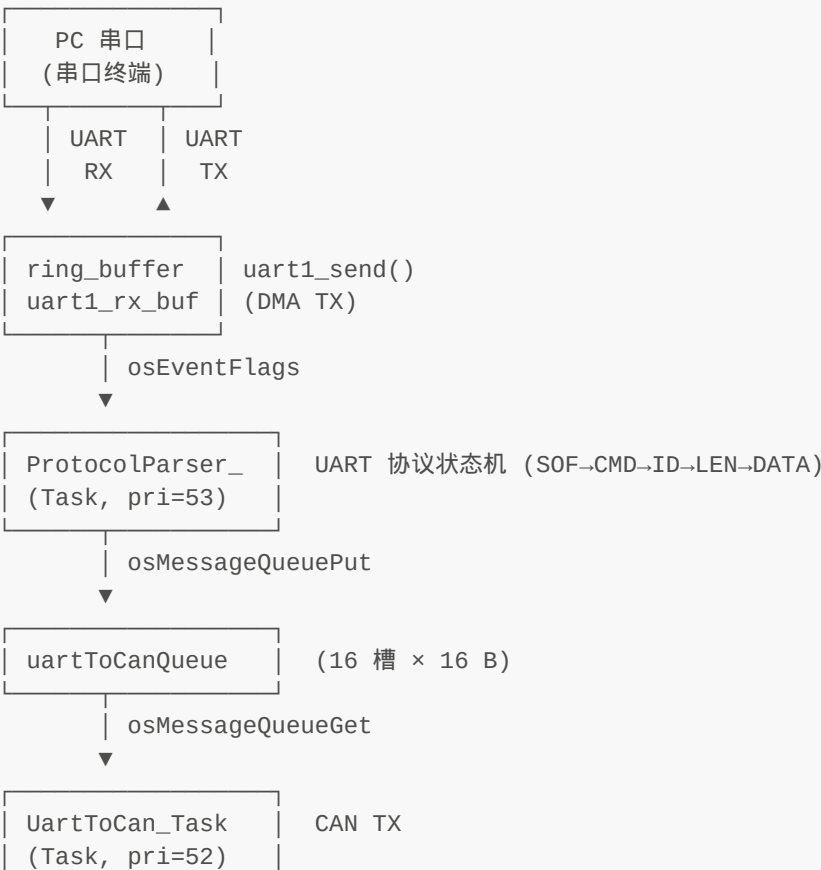
```
[PC 串口工具]
| 自定义帧: AA | CMD | ID(4B, LE) | DLC | Data
▼
USART1 外设 (中断接收)
|
▼ HAL_UART_RxCpltCallback(usart.c / ISR 上下文)
|  └─ ring_buffer_put(&uart1_rx_buffer, byte)      写 head
|  └─ osEventFlagsSet(uart1_rx_eventHandle, 0x01)  唤醒解析任务
▼
uart1_rx_buffer [RingBuffer_t, 256B] ← ISR 写 head / 任务读 tail, 无锁
|
▼ ProtocolParser_Task_Run(High 优先级)
| ring_buffer_get() 逐字节取出
| 状态机解析 → 组装 App_UART_Message_t
| osMessageQueuePut(uartToCanQueueHandle, &current_msg)
▼
uartToCanQueue [容量 16 × App_UART_Message_t]
|
▼ UartToCan_Task_Run(Normal 优先级)
| osMessageQueueGet() 阻塞等待
```



4.2 下行：CAN → UART



4.3 数据流总图





五、IPC 对象一览

对象	类型	容量	元素大小	生产者	消费者	用途
uart1_rx_buffer	RingBuffer	256 B	1 B	UART RxISR	ProtocolParser	字节级接收缓冲，无锁 SPSC
uart1_rx_eventHandle	EventFlags	32-bit	-	UART RxISR	ProtocolParser	新数据到达通知
uartToCanQueue	MessageQueue	16	App_UART_Message_t (16 B)	ProtocolParser	UartToCan	帧解析 → CAN 发送解耦
canRxQueue	MessageQueue	16	App_CAN_Message_t (16 B)	CAN RxISR	CanRxProcess	CAN 接收缓冲，ISR 快速返回
uart1_tx_mutexHandle	Mutex	1	-	-	uart1_send()	防多任务并发操作 UART1 TX
uart1_tx_semHandle	Semaphore	1	-	TxCplt 回调 ISR	uart1_send()	DMA 传输完成通知

App_UART_Message_t 与 App_CAN_Message_t 均为 16 字节（含对齐填充），队列内部按值拷贝传递。

六、关键同步机制汇总

同步对象	方向	生产者	消费者	解决问题
uart1_rx_buffer (RingBuffer)	ISR → Task	UART RxISR	ProtocolParser	字节接收缓冲，无锁 SPSC
uart1_rx_eventHandle (EventFlag)	ISR → Task	UART RxISR	ProtocolParser	事件驱动唤醒，零延迟
uartToCanQueue (Queue×16)	Task → Task	ProtocolParser	UartToCan	帧级解耦，背压保护
canRxQueue (Queue×16)	ISR → Task	CAN RxISR	CanRxProcess	CAN 报文缓冲，ISR 快速返回
uart1_tx_mutexHandle (Mutex)	多 → 单	—	uart1_send()	防两个任务同时操作 UART1 TX
uart1_tx_semHandle (Semaphore)	ISR → Task	TxCplt 回调	uart1_send()	DMA 完成通知，任务让出 CPU

七、初始化顺序

系统启动入口 main() 的执行序列：

```
main()
| 1. HAL_Init()           HAL 库初始化 (Systick 等)
| 2. SystemClock_Config() 72MHz: HSE(8MHz) → PLL(x9) → SYSCLK
| 3. MX_GPIO_Init()       GPIO 初始化
| 4. MX_DMA_Init()         DMA1: Ch4(USART1_TX) Ch5(USART1_RX)
|                           Ch6(USART2_RX) Ch7(USART2_TX)
| 5. MX_CAN_Init()         CAN1: 500kbps, Normal Mode, 自动重发
| 6. MX_USART1_UART_Init() USART1: 115200 8N1, DMA+IT
| 7. MX_USART2_UART_Init() USART2: 115200 8N1 (保留)
| 8. CAN_Filter_Config()   过滤器: ID=0x0000, Mask=0x0000 (全通)
| 9. HAL_CAN_Start()       启动 CAN 外设
|10. HAL_CAN_ActivateNotification() 使能 RX FIFO0 中断
|11. UART_Receive_Start()  初始化 ring_buffer + 启动 IT RX
|
|12. osKernelInitialize()  初始化 FreeRTOS 内核
|13. MX_FREERTOS_Init()    创建所有队列/互斥锁/信号量/事件/任务
|14. osKernelStart()       启动调度器 (不再返回)
|
▼
while(1) // 永不执行 - 控制权已交给 FreeRTOS
```